

Atelier 3: Signature numérique

Réalisé par : Audes Mariana MOUSSAVOU MOUSSAVOU

- Côté émetteur

1. Créer un fichier "message.txt" et y mettre un texte confidentiel
2. Générer les clés privées RSA de la source et la destination

...\Source> openssl genrsa -out src_rsa.pem -passout pass:srcpasswd -des3 512

```
DELL@MOUSSAVOU MINGW64 ~
$ cd "C:\Users\22891\Desktop\Programmation distribuée et sécurité\source"

DELL@MOUSSAVOU MINGW64 ~/Desktop/Programmation distribuée et sécurité/source
$ openssl genrsa -out src_rsa.pem -passout pass:srcpasswd -des3 512
```

...\Destinataire> openssl genrsa -out dest_rsa.pem -passout pass:destpasswd -des3 512

```
DELL@MOUSSAVOU MINGW64 ~
$ cd "C:\Users\22891\Desktop\Programmation distribuée et sécurité\destinataire"

DELL@MOUSSAVOU MINGW64 ~/Desktop/Programmation distribuée et sécurité/destinataire
$ openssl genrsa -out dest_rsa.pem -passout pass:destpasswd -des3 512
```

src_rsa.pem et **dest_rsa.pem** vont contenir les clés privées de la source et du destinataire protégés par les mots de passe srcpasswd et destpasswd respectivement.

3. Extraction des clés publiques à partir des fichiers "src_rsa.pem" et "dest_rsa.pem" vers "src_rsa_pub.pem" et "dest_rsa_pub.pem"

...\Source> openssl rsa -in src_rsa.pem -passin pass:srcpasswd -out src_rsa_pub.pem -pubout

```
DELL@MOUSSAVOU MINGW64 ~/Desktop/Programmation distribuée et sécurité/source
$ openssl rsa -in src_rsa.pem -passin pass:srcpasswd -out src_rsa_pub.pem -pubout
writing RSA key
```

 src_rsa.pem	15/03/2026 12:53	Fichier PEM	1 Ko
 src_rsa_pub.pem	15/03/2026 12:56	Fichier PEM	1 Ko

...\Destinataire> openssl rsa -in dest_rsa.pem -passin pass:destpasswd -out dest_rsa_pub.pem -pubout

```
DELL@MOUSSAVOU MINGW64 ~/Desktop/Programmation distribuée et sécurité/destinataire
$ openssl rsa -in dest_rsa.pem -passin pass:destpasswd -out dest_rsa_pub.pem -pubout
writing RSA key
```

☐ Nom	Modifié le	Type	Taille
 dest_rsa.pem	15/03/2026 12:59	Fichier PEM	1 Ko
 dest_rsa_pub.pem	15/03/2026 12:59	Fichier PEM	1 Ko

Ici l'option -pubout permet d'extraire la clé publique (par défaut c'est la clé privée qui est extraite) L'option passin et passout c'est pour protéger les fichiers, pass:xxxx permet de faire une protection par mots de passe.

4. échange de secret partagé

La source choisit un secret partagé (qui sera utilisé pour générer la clé symétrique partagée) par exemple "secret" (stocké dans secret.txt) et elle chiffre ce secret partagé avec la clé publique de la destination "dest_rsa_pub.pem" Ici "secret" sera utilisé par DES pour fabriquer la clé symétrique ET le vecteur d'initialisation nécessaire pour DES en mode CBC.

..\Source> openssl rsautl -in secret.txt -out secret.crypt -inkey dest_rsa_pub.pem -pubin -encrypt

```
DELL@MOUSSAVOU MINGW64 ~/Desktop/Programmation distribuée et sécurité/source
$ cp "C:\Users\22891\Desktop\Programmation distribuée et sécurité\destinaire\dest_rsa_pub.pem" "C:\Users\22891\Desktop\Programmation distribuée et sécurité\source"
DELL@MOUSSAVOU MINGW64 ~/Desktop/Programmation distribuée et sécurité/source
$ ls
dest_rsa_pub.pem message.txt secret.txt src_rsa.pem src_rsa_pub.pem
DELL@MOUSSAVOU MINGW64 ~/Desktop/Programmation distribuée et sécurité/source
$ openssl pkeyutl -encrypt -in secret.txt -pubin -inkey dest_rsa_pub.pem -out secret.crypt
```

5. Chiffrement de "message.txt" avec DES-CBC en utilisant le secret partagé "secret" comme paramètre de génération de clé de session et de vecteur d'initialisation.

...\Source> openssl enc -des-ede3-cbc -in message.txt -out message.crypt -pass file:secret.txt

```
DELL@MOUSSAVOU MINGW64 ~/Desktop/Programmation distribuée et sécurité/source
$ openssl enc -des-ede3-cbc -in message.txt -out message.crypt -pass file:secret.txt
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

6. Signature de message crypté "message.crypt"

Calcul de condensat (code de hashage) avec MD5 :

...\Source> openssl dgst -md5 -binary -out message.crypt.dgst message.crypt

```
DELL@MOUSSAVOU MINGW64 ~/Desktop/Programmation distribuée et sécurité/source
$ openssl dgst -md5 -binary -out message.crypt.dgst message.crypt
```

Chiffrement du condensat (code de hashage) avec la clé privée de la source "src_rsa.pem"

...\Source> openssl rsautl -in message.crypt.dgst -out message.crypt.dgst.sign -sign -inkey src_rsa.pem

```
DELL@MOUSSAVOU MINGW64 ~/Desktop/Programmation distribuée et sécurité/source
$ openssl rsautl -in message.crypt.dgst -out message.crypt.dgst.sign -sign -inkey src_rsa.pem
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.
Enter pass phrase for src_rsa.pem:
```

```
DELL@MOUSSAVOU MINGW64 ~/Desktop/Programmation distribuée et sécurité/source
$ |
```

ÁjMvâ±nñb&´)†{žÄäd@yI9ÅKA^ÖÄùn)E...‘·,_f)àHvÿÜ73«åi`mbÊêv...

Fichier Modifier Affichage

salted__@?r×ùH7<Ê>òë5S;aaYä 5«%±Gò

• Côté Récepteur

La destination connaît: dest_rsa.pem dest_rsa_pub.pem et reçoit: "secret.crypt" "message.crypt" "message.crypt.dgst.sign"

Donc il faudra recopier ces trois fichiers du répertoire Source vers le répertoire Destinataire.

1. Authentification de la source et de l'intégrité de donnée reçue.

Déchiffrement de l'empreinte (code de hashage) du message ==> dgst1

Programmation distribuée et sécurité | 2025

...\Destinataire> openssl rsautl -in message.crypt.dgst.sign -out dgst1 -pubin -inkey src_rsa_pub.pem

La destination calcule elle même le digest à partir de "message.crypt" ==> dgst2

...\Destinataire> openssl dgst -md5 -binary -out dgst2 message.crypt

```
DELL@MOUSSAVOU MINGW64 ~/Desktop/Programmation distribuée et sécurité/destinataire
$ openssl dgst -md5 -binary -out dgst2 message.crypt
```

La destination compare les deux condensats: (la commande FC fait un File Compare, /B fait une comparaison Binaire)

...\Destinataire> fc /B dgst1 dgst2

Comparaison des fichiers dgst1 et dgst2: aucune différence trouvée ==> donc l'intégrité du "message.crypt" est confirmée. ==> aussi l'authentification de la source est aussi vérifiée.

2. Déchiffrement du message "message.crypt"

Déchiffrement de la "clé partagée" (secret)

...\Destinataire> openssl rsautl -decrypt -in secret.crypt -out secret.txt -inkey dest_rsa.pem

```
DELL@MOUSSAVOU MINGW64 ~/Desktop/Programmation distribuée et sécurité/destinataire
$ openssl rsautl -decrypt -in secret.crypt -out secret.txt -inkey dest_rsa.pem
The command rsautl was deprecated in version 3.0. Use 'pkeyutl' instead.
Enter pass phrase for dest_rsa.pem:
```

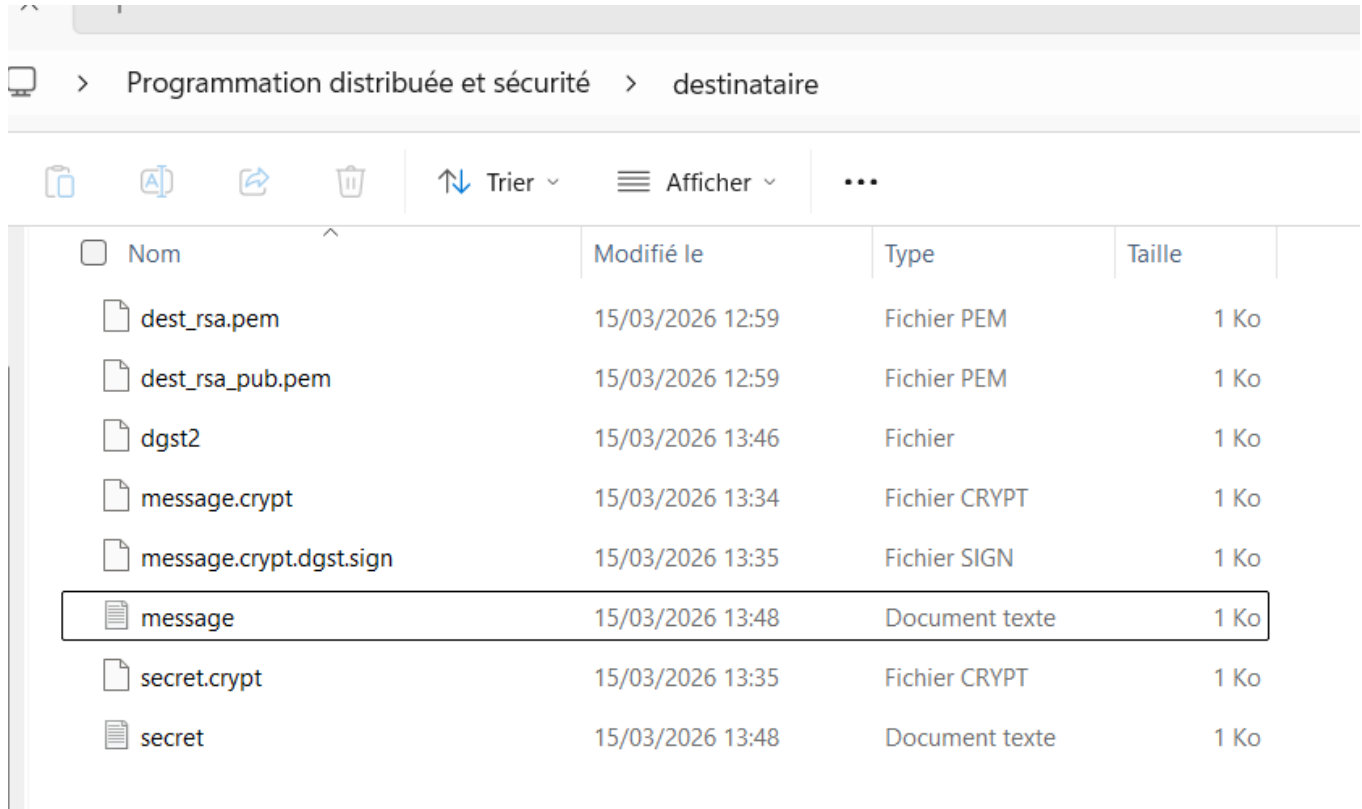
Déchiffrement du "message.crypt" à l'aide de secret qui se trouve dans "secret.txt"

...\Destinataire> openssl enc -in message.crypt -out message.txt -pass file:secret.txt -d -des-ede3-cbc

```
DELL@MOUSSAVOU MINGW64 ~/Desktop/Programmation distribuée et sécurité/destinataire
$ openssl enc -in message.crypt -out message.txt -pass file:secret.txt -d -des-ede3-cbc
*** WARNING : deprecated key derivation used.
Using -iter or -pbkdf2 would be better.
```

Lire le message! ! !

...\Destinataire> type message.txt



Nom	Modifié le	Type	Taille
dest_rsa.pem	15/03/2026 12:59	Fichier PEM	1 Ko
dest_rsa_pub.pem	15/03/2026 12:59	Fichier PEM	1 Ko
dgst2	15/03/2026 13:46	Fichier	1 Ko
message.crypt	15/03/2026 13:34	Fichier CRYPT	1 Ko
message.crypt.dgst.sign	15/03/2026 13:35	Fichier SIGN	1 Ko
message	15/03/2026 13:48	Document texte	1 Ko
secret.crypt	15/03/2026 13:35	Fichier CRYPT	1 Ko
secret	15/03/2026 13:48	Document texte	1 Ko

Service confidentialité assuré